

PromptLab — Memoria del Proyecto

Asignatura: Aplicaciones para Lenguajes y Sistemas

Curso: 2025/2026

Autor: Alejandro López Gómez

DNI: 32723985E

URL: <https://als-promptlab-production.up.railway.app/>

Repositorio: <https://github.com/AlexLopezGomez/ALS-PromptLab>

UniversidadeVigo

PromptLab — Memoria del Proyecto	1
1. Introducción	3
2. Modelo de dominio	4
2.1. Entidades	4
2.2. Relaciones	4
3. Arquitectura y organización del código	6
4. Persistencia con Sirope sobre Redis	8
5. Integridad referencial y borrado en cascada	9
5.1. Borrado en cascada — Prompt	9
5.2. Desasociación — Categoria	9
5.3. Borrado simple — Coleccion	9
5.4. Carga defensiva	9
6. Autenticación y control de acceso	10
7. Interfaz de usuario	11
7.1. Navegación	11
7.2. Front-end	11
7.3. Formularios y mensajes	11
8. Robustez y manejo de errores	12
9. Guía de instalación y ejecución	13
9.1. Requisitos previos	13
9.2. Instalación de dependencias	13
9.3. Ejecución	13
9.4. Datos de demostración (opcional)	13
9.5. Variables de entorno	14
10. Manual de usuario	15
11. Decisiones de diseño	17
12. Posibles mejoras	18
13. Conclusiones	19

1. Introducción

PromptLab es una aplicación web comunitaria desarrollada con Flask que permite a sus usuarios crear, organizar, comentar y evaluar *prompts* destinados a modelos de inteligencia artificial. El propósito de la plataforma es centralizar la gestión del conocimiento sobre técnicas de prompting: cualquier usuario registrado puede publicar un prompt, enriquecerlo con etiquetas y categorías, agrupar sus mejores prompts en colecciones temáticas, y recibir comentarios y puntuaciones del resto de la comunidad.

Desde el punto de vista técnico, el proyecto implementa todos los requisitos del enunciado: aplicación web con Flask, plantillas Jinja2, autenticación mediante Flask-Login y persistencia a través de Sirope sobre Redis. Se han modelado cinco entidades de dominio además del usuario, lo que sitúa el proyecto en la franja de mayor complejidad según el criterio de evaluación.

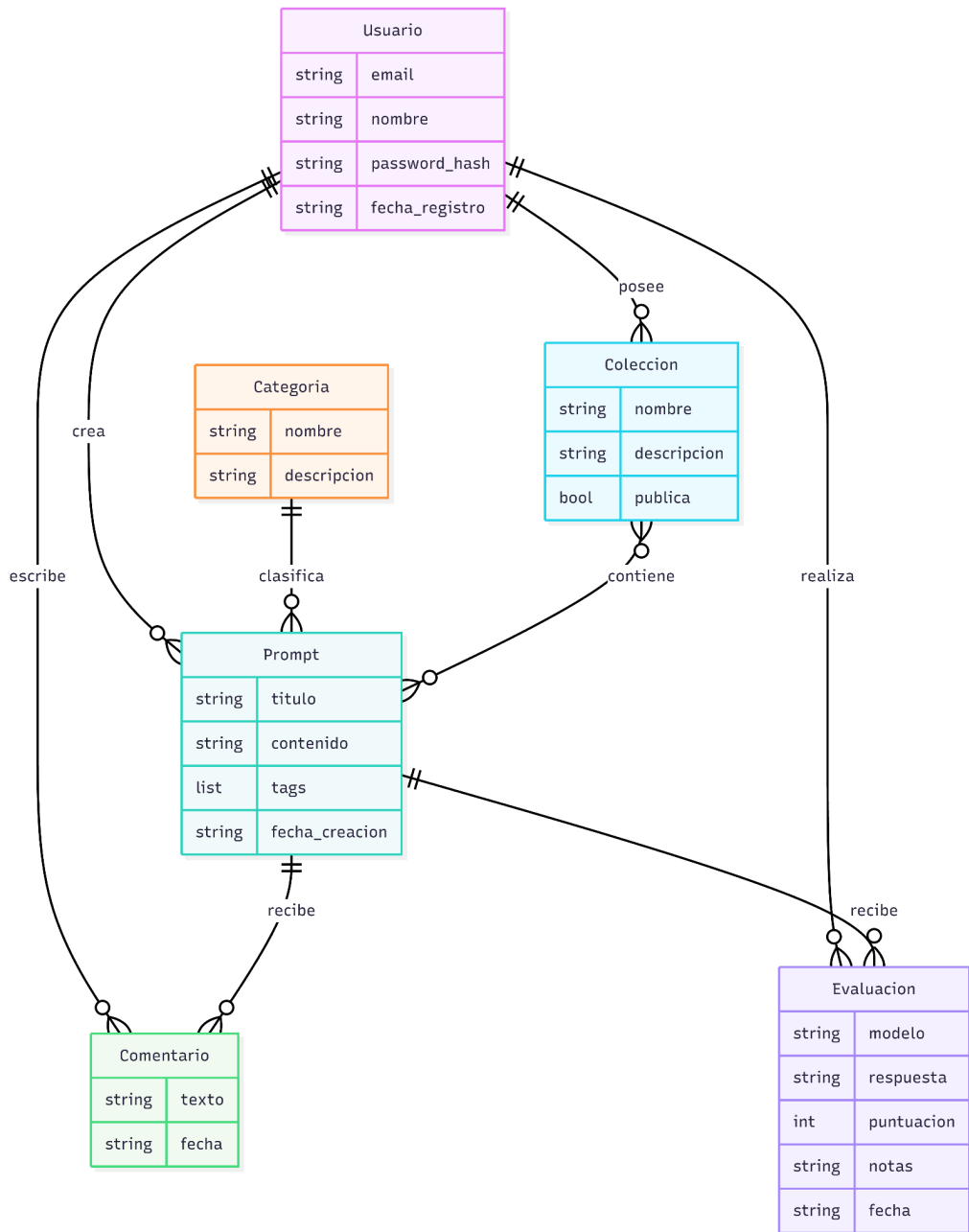
2. Modelo de dominio

2.1. Entidades

El dominio se representa mediante cinco clases persistentes, almacenadas en el paquete `models/`. El usuario (*Usuario*) actúa como entidad de infraestructura y no se contabiliza como entidad de dominio.

Entidad	Responsabilidad
Prompt	Texto principal: título, contenido, categoría opcional y etiquetas libres.
Categoría	Clasificación temática compartida por todos los usuarios.
Colección	Agrupación personal de prompts; puede ser pública o privada.
Comentario	Mensaje libre de un usuario asociado a un prompt concreto.
Evaluación	Puntuación de 1 a 5 estrellas con texto opcional sobre un prompt.

2.2. Relaciones



Un Usuario puede ser autor de N Prompt, N Colección, N Comentario y N Evaluación. Un Prompt pertenece a 0..1 Categoría y acumula N Comentario y N Evaluación. Una Colección referencia M Prompt mediante una lista de OIDs, modelando una relación N a N sin tabla intermedia.

Todas las relaciones se implementan mediante referencias por OID, nunca por inclusión directa de objetos, lo que garantiza que el borrado y la carga de cada entidad sean independientes y transaccionalmente seguros.

3. Arquitectura y organización del código

La aplicación aplica el principio de un módulo por entidad en todas las capas, de forma que cada entidad define su modelo, sus rutas y sus plantillas en ficheros separados:

src/

- |— app.py
- |— sirope_config.py
- |— requirements.txt
- |— models/
 - | |— usuario.py
 - | |— prompt.py
 - | |— categoria.py
 - | |— coleccion.py
 - | |— comentario.py
 - | |— evaluacion.py
- |— routes/
 - | |— auth.py
 - | |— prompts.py
 - | |— categorias.py
 - | |— colecciones.py
 - | |— comentarios.py
 - | |— evaluaciones.py
- |— templates/
 - | |— base.html
 - | |— dashboard.html
 - | |— auth/
 - | |— prompts/

- | |— colecciones/
- | |— categorias/
- | |— evaluaciones/
- | |— errores/
- |— static/css/custom.css

Cada entidad expone de manera uniforme una clase de modelo con constructor explícito y propiedad `oid`, funciones auxiliares de carga, búsqueda, filtrado y borrado en cascada, un blueprint de Flask con todas las rutas CRUD asociadas, y sus plantillas Jinja2 en `templates/<entidad>/`.

La función `crear_app()` en `app.py` actúa como fábrica: instancia Flask, registra los seis blueprints, configura Flask-Login, define la ruta raíz y el dashboard, registra los manejadores de error 404 y 500, y añade el filtro Jinja `fechabonita` para formatear fechas ISO 8601.

4. Persistencia con Sirope sobre Redis

El módulo `sirope_config.py` centraliza la creación de la instancia de Sirope y su conexión a Redis. La configuración se resuelve por prioridad:

1. Variable de entorno `REDIS_URL` (formato Railway/Heroku).
2. Variables individuales `REDIS_HOST`, `REDIS_PORT` y `REDIS_DB` para desarrollo local (valores por defecto: `localhost:6379`, base de datos 0).

El módulo expone además dos funciones de utilidad: `safe_oid(srp, obj)`, que serializa el OID interno de Sirope a una cadena segura para usar en URLs; y `oid_from_safe(srp, cadena)`, que reconstruye el OID a partir de su representación segura.

El uso sistemático de *safe OIDs* en todas las rutas evita exponer los identificadores internos de Redis en la barra de direcciones y produce URLs estables que pueden compartirse directamente. Cada petición HTTP crea una instancia nueva de Sirope, eliminando la posibilidad de estados compartidos entre módulos.

5. Integridad referencial y borrado en cascada

La ausencia de un motor relacional obliga a gestionar de forma explícita la coherencia entre entidades relacionadas. Se han adoptado tres estrategias distintas según la naturaleza de la relación.

5.1. Borrado en cascada — Prompt

Al eliminar un Prompt (`borrar_prompt_en_cascada` en `models/prompt.py`): se borran todas sus Evaluación asociadas, se borran todos sus Comentario asociados, se elimina su OID de la lista `prompt_oids` de cualquier Colección que lo referenciara, y finalmente se borra el Prompt en sí.

5.2. Desasociación — Categoría

Al borrar una Categoría, no se destruyen los prompts que la referenciaban. En su lugar, `desasociar_prompts_de_categoria` pone `categoria_oid = None` en cada uno de ellos antes de eliminar la categoría. De este modo el contenido del usuario no se pierde.

5.3. Borrado simple — Colección

Eliminar una Colección solo borra el agrupamiento; los Prompt que contenía no se ven afectados.

5.4. Carga defensiva

Toda carga de un objeto relacionado mediante su OID se envuelve en `try/except`. Un OID huérfano nunca produce un error en la vista: el valor se sustituye por `None` y la plantilla muestra un valor por defecto como *"Autor desconocido"*.

6. Autenticación y control de acceso

La autenticación se gestiona íntegramente con Flask-Login. La clase Usuario implementa UserMixin y sobrescribe get_id() para devolver el *safe OID* del usuario, que actúa como identificador de sesión. Las contraseñas se almacenan únicamente en forma de hash generado con werkzeug.security.generate_password_hash (algoritmo scrypt por defecto en Werkzeug 3.x). Todas las operaciones de escritura están protegidas con el decorador @login_required.

El control de autorización aplica las siguientes reglas de negocio:

Acción	Condición
Editar o borrar un Prompt	Solo su autor.
Editar o borrar una Evaluacion	Solo su autor.
Borrar un Comentario	El autor del comentario o el autor del prompt al que pertenece.
Editar o borrar una Coleccion	Solo su propietario.
Ver una Coleccion privada	Solo su propietario.

7. Interfaz de usuario

7.1. Navegación

La barra de navegación superior, presente en todas las páginas mediante `base.html`, proporciona acceso directo a Explorar, Categorías, Mis Prompts, Mis Colecciones, Dashboard y las opciones de sesión. Cada vista incluye migas de pan (*breadcrumbs*) que permiten retroceder a cualquier nivel de la jerarquía sin recurrir al botón «atrás» del navegador. Tras cualquier operación de creación, edición o borrado se aplica el patrón POST/Redirect/GET y se muestra un mensaje *flash* con el resultado.

7.2. Front-end

Se utiliza Pico CSS v2 (cargado desde CDN) como sistema de diseño base: tipografía, botones, formularios y cuadrícula *responsive* sin configuración adicional. El fichero `static/css/custom.css` añade estilos propios para la barra de navegación, tarjetas de prompt, etiquetas, mensajes flash y elementos específicos de cada vista. La plantilla `base.html` define bloques `titulo`, `breadcrumbs` y `content` que heredan el resto de plantillas, garantizando coherencia visual y evitando duplicación de estructura HTML.

7.3. Formularios y mensajes

Los formularios validan los campos obligatorios en el servidor antes de persistir. Si la validación falla, el formulario se re-renderiza con los valores introducidos y un mensaje de error. Las plantillas `errores/404.html` y `errores/500.html` presentan páginas de error con enlace de retorno al inicio.

8. Robustez y manejo de errores

La aplicación aplica una estrategia de defensa en profundidad. En la capa de persistencia, todas las llamadas a Sirope se envuelven en try/except; en caso de fallo se devuelve una lista vacía o None, nunca se propaga la excepción a la vista. En la capa de rutas, los OID seguros recibidos en la URL se validan antes de usarlos; si no resuelven a un objeto, la ruta aborta con 404. Los manejadores globales `@app.errorhandler(404)` y `@app.errorhandler(500)` muestran páginas de error informativas sin exponer trazas al usuario. Los campos obligatorios se comprueban explícitamente antes de cualquier operación de escritura, y no se comparte estado entre blueprints.

9. Guía de instalación y ejecución

9.1. Requisitos previos

- Python 3.10 o superior.
- Redis instalado y accesible en localhost:6379 (o configurable mediante variables de entorno).

9.2. Instalación de dependencias

```
python -m venv .venv
```

```
source .venv/bin/activate    # Linux / macOS
```

```
.venv\Scripts\activate      # Windows (PowerShell)
```

```
pip install -r src/requirements.txt
```

9.3. Ejecución

```
# Terminal 1 — arrancar Redis
```

```
redis-server
```

```
# Terminal 2 — arrancar la aplicación Flask
```

```
flask run
```

La aplicación queda disponible en <http://127.0.0.1:5000/>.

9.4. Datos de demostración (opcional)

El script `src/seed.py` puebla la base de datos con usuarios, prompts, categorías, colecciones, comentarios y evaluaciones de ejemplo. Es idempotente: si alguno de los objetos ya existe, lo omite sin duplicarlo.

```
cd src
```

```
python seed.py
```

Cuentas disponibles tras ejecutarlo:

Email	Contraseña
ana@demo.com	demo1234
luis@demo.com	demo1234

9.5. Variables de entorno

Variable	Valor por defecto	Descripción
SECRET_KEY	<i>(valor de desarrollo)</i>	Clave secreta de Flask para firmar la sesión.
REDIS_URL	—	URL completa de Redis (tiene prioridad sobre las siguientes).
REDIS_HOST	localhost	Host del servidor Redis.
REDIS_PORT	6379	Puerto del servidor Redis.
REDIS_DB	0	Número de base de datos de Redis.

10. Manual de usuario

Funcionalidad	Descripción
Registro	Acceder a /auth/registro e introducir nombre, email y contraseña (mínimo 6 caracteres).
Inicio de sesión	Acceder a /auth/login con email y contraseña registrados.
Dashboard	Vista resumen con el número de prompts y colecciones propios, y los cinco últimos prompts de la comunidad.
Explorar prompts	Listado público con buscador por título y etiqueta. Accesible sin sesión.
Crear prompt	Desde el botón «Nuevo prompt» en el menú o en «Mis prompts». Campos: título, contenido, categoría opcional y etiquetas separadas por coma.
Detalle de un prompt	Muestra contenido, autor, categoría, etiquetas, evaluaciones y sección de comentarios.
Evaluaciones	Formulario con modelo de IA utilizado, texto de respuesta obtenida, puntuación (1–5 estrellas) y notas opcionales.
Colecciones	Crear colecciones públicas o privadas, añadir o quitar prompts, y ver las colecciones públicas de otros usuarios.

Categorías	Cualquier usuario autenticado puede crear, editar o eliminar categorías. Al borrar una, los prompts asociados pasan a quedar sin categoría.
-------------------	---

11. Decisiones de diseño

OID seguros en URLs. Nunca se expone el identificador interno de Sirope en la URL. Todas las rutas usan la representación segura devuelta por `srp.safe_from_oid()`, que produce cadenas opacas y estables aptas para compartir y marcar como favoritas.

Carga defensiva generalizada. Cualquier resolución de un OID se realiza dentro de un bloque `try/except`. Esto garantiza que referencias huérfanas no propaguen excepciones a la vista ni rompan el listado.

Estrategia diferenciada de borrado. Se distingue entre entidades *contenedoras* (Prompt) y entidades *clasificadoras* (Categoría). Borrar un prompt destruye todo su contenido derivado, mientras que borrar una categoría solo desasocia los prompts sin eliminarlos, preservando el trabajo del usuario.

Patrón POST/Redirect/GET. Toda mutación del estado termina con una redirección HTTP, de modo que recargar la página no reenvía el formulario. Los mensajes flash comunican el resultado en la vista de destino.

Sin estado entre peticiones. Cada petición instancia su propia conexión a Sirope. No existe ningún objeto global mutable compartido entre blueprints.

12. Posibles mejoras

- API REST/JSON paralela a la interfaz web, para consumo desde clientes externos.
- Roles de administración con capacidad de moderación global.
- Búsqueda por contenido completo del prompt, no solo por título y etiquetas.
- Importación y exportación de colecciones en formato JSON.
- Tests automáticos con pytest y una instancia de Redis en memoria (fakeredis).
- Paginación en los listados de prompts y colecciones.

13. Conclusiones

PromptLab cumple íntegramente los requisitos técnicos y funcionales del proyecto: cinco entidades de dominio persistidas con Sirope/Redis, autenticación segura con Flask-Login, interfaz web construida con plantillas Jinja2, organización modular con un blueprint por entidad, navegación coherente sin necesidad del botón «atrás», validación de formularios con retroalimentación al usuario, integridad referencial con borrado en cascada y desasociación selectiva, y manejo de errores en todas las capas.

La arquitectura basada en blueprints permite añadir nuevas entidades sin modificar el código existente. La separación estricta entre modelo, rutas y plantillas facilita la localización de cualquier componente y reduce el acoplamiento entre módulos. El uso de *safe OIDs* en las URLs y la estrategia de carga defensiva hacen que la aplicación sea robusta ante situaciones de datos inconsistentes derivadas de operaciones de borrado concurrentes o parciales.